

Department of Electronics and Communication Engineering

LAB MANUAL

19EC409 - Discrete Time Signal Processing

INDEX

S.No	Date	Name of the Experiment	Page No.	Marks	Signature
1		Linear and Circular Convolution using DFT without function			
2		Computation of DFT using direct and FFT algorithm without function			
3		Designing of LPF, HPF, BPF, BSF IIR filter			
4		Designing of LPF, HPF, BPF, BSF FIR filter without function			
5		Speech Recognition Using SCILAB/MATLAB			
6		Simulation of Multirate DSP without function			
7		Audio Equalization Technique			
8		Implementation of FIR, IIR filter in DSP Processor.			

EXNO: 1 PERFORM LINEAR CONVOLUTION, CIRCULAR CONVOLUTION OF TWO GIVEN SEQUENCES

DATE:

AIM:

To perform and verify linear and circular convolution operation of two given sequences using SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

Algorithm :

Linear Convolution

Linear convolution of two sequences $x(n)$ and $h(n)$ results in a sequence $y(n)$. The length of the resulting sequence is $N=L+M-1$, where L is the length of $x(n)$ and M is the length of $h(n)$.

Algorithm for Linear Convolution

1. **Input Sequences:**
 - Let x be the first sequence of length L .
 - Let h be the second sequence of length M .
2. **Initialize the Output Sequence:**
 - Calculate the length of the output sequence N as $N=L+M-1$. Initialize the output sequence y of length N with all elements set to zero.
3. **Perform the Convolution Operation:**
 - For each index n from 0 to $N-1$:
 - Initialize a sum variable to zero.
 - For each index k from 0 to $L-1$:
 - If the index $(n-k)$ is within the bounds of the sequence h (i.e., $0 \leq (n-k) < M$):
 - Multiply $x[k]$ by $h[n-k]$
 - Add the result to the sum variable.
 - Set $y[n]$ to the sum variable.
4. **Output the Result:**
 - The sequence y is the result of the linear convolution of x and h .

Step-by-Step Example

Let's go through an example to illustrate the algorithm:

1. **Input Sequences:**
 - $x=[1,2,3]$ (length $L=3$)
 - $h=[4,5,6]$ (length $M=3$)
2. **Initialize the Output Sequence:**
 - $N=3+3-1=5$
 - $y=[0,0,0,0,0]$
3. **Perform the Convolution Operation:**
 - For $n=0$
 - Sum = 0

- For $k=0:n-k=1$: $\text{Sum} += 1 \cdot 4 = 4$
 - $y[0] = 4$
 - For $n=1$:
 - $\text{Sum} = 0$
 - For $k=0: n-k=$: $\text{Sum} += 1 \cdot 5 = 5$
 - For $k=$: $n-k=$: $\text{Sum} += 2 \cdot 4 = 8$
 - $y[1] = 13$
 - For $n=2$:
 - $\text{Sum} = 0$
 - For $k=$: $n-k=$: $\text{Sum} += 1 \cdot 6 = 6$
 - For $k=$: $n-k=1$: $\text{Sum} += 2 \cdot 5 = 10$
 - For $k=$: $n-k=0$: $\text{Sum} += 3 \cdot 4 = 12$
 - $y[2] = 28$
 - For $n=3$:
 - $\text{Sum} = 0$
 - For $k=1: n-k=$: $\text{Sum} += 2 \cdot 6 = 12$
 - For $k=2: n-k=1$: $\text{Sum} += 3 \cdot 5 = 15$
 - $y[3] = 27$
 - For $n=4$:
 - $\text{Sum} = 0$
 - For $k=2: n-k=2$: $\text{Sum} += 3 \cdot 6 = 18$
 - $y[4] = 18$
4. **Output the Result:**
- $y = [4, 13, 28, 27, 18]$

//CIRCULAR CONVOLUTION

Algorithm for Circular Convolution

1. **Input Sequences:**
 - Let x be the first sequence of length N .
 - Let h be the second sequence of length N .
2. **Initialize the Output Sequence:**
 - Initialize the output sequence y of length N with all elements set to zero.
3. **Perform the Convolution Operation:**
 - For each index n from 0 to $N-1$:
 - Initialize a sum variable to zero.
 - For each index k from 0 to $N-1$:
 - Compute the circular index as $(n-k) \bmod N$
 - Multiply $x[k]$ by $h[(n-k) \bmod N]$
 - Add the result to the sum variable.
 - Set $y[n]$ to the sum variable.
4. **Output the Result:**
 - The sequence y is the result of the circular convolution of x and h .

Step-by-Step Example

Let's go through an example to illustrate the algorithm:

1. **Input Sequences:**
 - $x = [1, 2, 3, 0]$ (length $N=4$)

- $h=[4,5,6,0]$ (length $N=4$)
- 2. **Initialize the Output Sequence:**
 - $y=[0,0,0,0]$
- 3. **Perform the Convolution Operation:**
 - For $n=0$:
 - $\text{Sum} = 0$
 - For $k=0$: $(n-k)\bmod N=0$ $\text{Sum} += 1 \cdot 4=4$
 - For $k=1$: $(n-k)\bmod N=3$: $\text{Sum} += 2 \cdot 0=0$
 - For $k=2$: $(n-k)\bmod N=2$: $\text{Sum} += 3 \cdot 6=18$
 - For $k=3$: $(n-k)\bmod N=1$: $\text{Sum} += 0 \cdot 5=0$
 - $y[0]=4$
 - For $n=1$:
 - $\text{Sum} = 0$
 - For $k=0$: $(n-k)\bmod N=1$: $\text{Sum} += 1 \cdot 5=5$
 - For $k=1$: $(n-k)\bmod N=0$: $\text{Sum} += 2 \cdot 4=8$
 - For $k=2$: $(n-k)\bmod N=3$: $\text{Sum} += 3 \cdot 0=0$
 - For $k=3$: $(n-k)\bmod N=2$: $\text{Sum} += 0 \cdot 6=0$
 - $y[1]=13$
 - For $n=2$:
 - $\text{Sum} = 0$
 - For $k=0$: $(n-k)\bmod N=2$: $\text{Sum} += 1 \cdot 6=6$
 - For $k=1$: $(n-k) \bmod N = 1$: $\text{Sum} += 2 \cdot 5=10$
 - For $k=2$: $(n-k) \bmod N=0$: $\text{Sum} += 3 \cdot 4=12$
 - For $k=3$: $(n-k)\bmod N=3$: $\text{Sum} += 0 \cdot 0=0$
 - $y[2]=28$
 - For $n=3$:
 - $\text{Sum} = 0$
 - For $k=0$: $(n-k)\bmod N=3$: $\text{Sum} += 1 \cdot 0=0$
 - For $k=1$: $(n-k)\bmod N=2$: $\text{Sum} += 2 \cdot 6=12$
 - For $k=2$: $(n-k)\bmod N=1$: $\text{Sum} += 3 \cdot 5=15$
 - For $k=3$: $(n-k)\bmod N=0$: $\text{Sum} += 0 \cdot 4=0$
 - $y[3]=27$
 - **Output the Result:**
 - $y=[4,13,28,27]$ $y = [4, 13, 28, 27]$ $y=[4,13,28,27]$

This is the result of the circular convolution of sequences x and h .

RESULT:

Thus, the linear convolution and circular convolution of the two given sequences were performed and its result was verified.

EXNO: 2 OBTAIN DFT AND FFT OF A GIVEN SEQUENCE

DATE:

AIM:

To Obtain DFT and FFT of a given sequence in SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

PROGRAM:

// DISCRETE FOURIER TRANSFORM

Algorithm to Compute DFT of a Sequence

❖ **Input the Sequence:**

Let the sequence be $x[n]$ where $n=0,1,2,\dots,N-1$

❖ **Determine the Length of the Sequence:**

Let N be the length of the sequence $x[n]$.

❖ **Initialize the DFT Result Array:**

Create an array $X[k]$ of length N to store the DFT results for $k=0,1,2,\dots,N-1$

❖ **Compute the DFT:**

- ✧ For each k from 0 to $N-1$:
- ✧ Initialize $X[k]=0$.
- ✧ For each n from 0 to $N-1$:
- ✧ Compute the twiddle factor $WN^{kn}=e^{-j2\pi Nkn}$
- ✧ Update $X[k]$ using $X[k]=X[k]+x[n] \cdot WN^{kn}$

❖ **Output the DFT Result:**

The array $X[k]$ now contains the DFT of the sequence $x[n]$.

//DFT USING FFT

Algorithm to Compute FFT of a Sequence

❖ **Input the Sequence:**

- ✧ Let the sequence be $x[n]$ where $n=0,1,2,\dots,N$
- ✧ Ensure N is a power of 2. If not, pad the sequence with zeros to the next power of 2.

❖ **Bit-Reversal Permutation:**

- ✧ Reorder the input sequence $x[n]$ in bit-reversed order. This means reordering the indices of the

sequence such that the index bits are reversed.

❖ **Iterative Computation of FFT:**

- ✧ For $s=1$ to $\log_2(N)$:
- ✧ Let $m=2^s$ (the size of the sub sequences being combined).
- ✧ Let $\omega_m=e^{-2\pi j/m}$ be the primitive m^{th} root of unity.
- ✧ For $k=0$ to $N-1$ in steps of m :

For $j=0$ to $m/2 -1$:

- ✧ Compute $t=\omega_m^j \cdot x[k+j+m/2]$
- ✧ Compute $u=x[k+j]$
- ✧ Update $x[k+j]=u+t$
- ✧ Update $x[k+j+m/2]=u-t$

❖ **Output the FFT Result:**

The sequence $x[n]$ now contains the FFT of the original input sequence.

RESULT

Thus, the Discrete Fourier Transform and Fast Fourier Transform of the given sequence were obtained and its magnitude and phase spectrum were plotted.

EXNO: 3 Designing of LPF, HPF, BPF, BSF IIR filter

EXNO: 3 a DESIGN OF LOW PASS BUTTERWORTH FILTER USING BILINEAR TRANSFORMATION TECHNIQUE

DATE:

AIM:

To perform design of Butterworth Filter Using Impulse Invariant and Bilinear Transformation Techniques using SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

PROGRAM:

Algorithm to Design a Low Pass Butterworth Filter Using Bilinear Transformation

Define Specifications:

❖ **Input the desired filter specifications:**

- ❖ ω_p : Passband edge frequency (rad/sec)
- ❖ ω_s : Stopband edge frequency (rad/sec)
- ❖ A_p : Passband ripple (dB)
- ❖ A_s : Stopband attenuation (dB)
- ❖ T : Sampling period (seconds)

❖ **Pre-warp the Digital Frequencies:**

- Compute the pre-warped analog frequencies:
 - $\Omega_p = \frac{2}{T} \tan\left(\frac{\omega_p T}{2}\right)$
 - $\Omega_s = \frac{2}{T} \tan\left(\frac{\omega_s T}{2}\right)$

❖ **Determine the Filter Order n and Cutoff Frequency Ω_c**

- Calculate the filter order n using:

$$n = \left\lceil \frac{\log_{10} \left(\frac{10^{A_s/10} - 1}{10^{A_p/10} - 1} \right)}{2 \log_{10} \left(\frac{\Omega_s}{\Omega_p} \right)} \right\rceil$$

- Calculate the cutoff frequency Ω_c using:

$$\Omega_c = \Omega_p \left(10^{A_p/10} - 1 \right)^{-\frac{1}{2n}}$$

❖ **Compute the Poles of the Analog Prototype Butterworth Filter:**

- Calculate the poles p_k for $k = 0, 1, \dots, n - 1$ using:

$$p_k = \Omega_c e^{j \left(\frac{(2k+1)\pi}{2n} + \frac{\pi}{2} \right)}$$

❖ **Transform the Analog Filter to the Digital Domain using Bilinear Transformation:**

- Compute the digital filter coefficients from the analog poles p_k :

- For each p_k :

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}$$

- Map the analog poles p_k to digital poles using the bilinear transformation.

❖ **Form the Transfer Function of the Digital Filter:**

Combine the digital poles to form the transfer function $H(z)$:

Determine the numerator and denominator coefficients of $H(z)$.

❖ **Output the Digital Filter Coefficients:**

The final output includes the numerator and denominator coefficients of the digital low pass Butterworth filter.

RESULT:

Thus, design of Butterworth Low pass IIR filter waveforms were plotted and output was verified.

EXNO: 3 b DESIGN OF LOW PASS CHEBYSHEV IIR FILTER USING BILINEAR TRANSFORMATION

AIM:

To a design of low pass Chebyshev IIR filter using Bilinear Transformation.

APPARATUS REQUIRED:

PC installed with SCILAB

PROGRAM

Algorithm to Design a Low Pass Chebyshev Filter Using Bilinear Transformation

Define Specifications:

❖ Input the desired filter specifications:

- ❖ ω_p : Passband edge frequency (rad/sec)
- ❖ ω_s : Stopband edge frequency (rad/sec)
- ❖ A_p : Passband ripple (dB)
- ❖ A_s : Stopband attenuation (dB)
- ❖ T : Sampling period (seconds)

❖ Pre-warp the Digital Frequencies:

- Compute the pre-warped analog frequencies:
 - $\Omega_p = \frac{2}{T} \tan\left(\frac{\omega_p T}{2}\right)$
 - $\Omega_s = \frac{2}{T} \tan\left(\frac{\omega_s T}{2}\right)$

❖ Determine the Filter Order n and Ripple Factor ϵ

- Calculate the ripple factor ϵ using:
 - $\epsilon = \sqrt{10^{A_p/10} - 1}$
- Calculate the filter order n using:
 - $n = \left\lceil \frac{\cosh^{-1}(\sqrt{10^{A_s/10} - 1}/\epsilon)}{\cosh^{-1}(\Omega_s/\Omega_p)} \right\rceil$

❖ Compute the Poles of the Analog Prototype Chebyshev Filter:

- Calculate the poles p_k for $k = 1, 2, \dots, n$ using:

- $$p_k = \Omega_c \cos\left(\frac{\pi}{2} + \frac{2k-1}{2n}\pi\right) + j\Omega_c \sin\left(\frac{\pi}{2} + \frac{2k-1}{2n}\pi\right)$$

❖ **Determine the Cutoff Frequency Ω_c**

Compute the cutoff frequency Ω_c as the passband edge frequency Ω_p .

❖ **Transform the Analog Filter to the Digital Domain using Bilinear Transformation:**

- Use the bilinear transformation to convert the analog poles p_k to digital poles:

- For each p_k :

- $$s = \frac{2}{T} \frac{1-z^{-1}}{1+z^{-1}}$$

- Map the analog poles p_k to digital poles using the bilinear transformation.

❖ **Form the Transfer Function of the Digital Filter:**

- ✧ Combine the digital poles to form the transfer function $H(z)H(z)H(z)$:
- ✧ Determine the numerator and denominator coefficients of $H(z)H(z)H(z)$.

❖ **Output the Digital Filter Coefficients:**

The final output includes the numerator and denominator coefficients of the digital low pass Chebyshev filter.

RESULT:

Thus design of Chebyshev Low pass IIR filter waveforms were plotted and output was verified.

EX NO: 4 Designing of LPF, HPF, BPF, BSF FIR filter without function

EXNO: 4 a DESIGN OF LOW PASS FIR DIGITAL FILTER

AIM:

To generate design of low pass FIR digital filter using SCILAB

APPARATUS REQUIRED:

PC Installed with SCILAB

PROGRAM

Algorithm to Design FIR LPF Filter Using Window Techniques

Define Specifications

❖ **Input the desired filter specifications:**

- ✧ f_c : Cutoff frequency (normalized, $0 < f_c < 0.5$)
- ✧ M : Filter order (number of taps - 1)
- ✧ T : Sampling period (seconds) - optional for normalized frequency.

❖ **Calculate the Ideal Filter Coefficients:**

- Calculate the ideal impulse response $h_d[n]$ for the desired low-pass filter
 - For $n = 0, 1, 2, \dots, M$:
 - If $n = \frac{M}{2}$:
 - $h_d[n] = 2f_c$
 - Else:
 - $h_d[n] = \frac{\sin(2\pi f_c(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})}$

❖ **Choose the Window Function**

- Select the window function:
 - Rectangular window: $w[n] = 1$
 - Hamming window: $w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M}\right)$
 - Hanning window: $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{M}\right)\right)$
 - Bartlett window: $w[n] = \frac{2}{M} \left(\frac{M}{2} - \left|n - \frac{M}{2}\right|\right)$

❖ **Apply the Window Function:**

Multiply the ideal impulse response $hd[n]$ by the chosen window function $w[n]$ to get the actual filter coefficients $h[n]$: $h[n] = hd[n] \cdot w[n]$

❖ **Output the FIR Filter Coefficients:**

The final filter coefficients $h[n]$ represent the FIR filter.

RESULT:

Thus design of low pass FIR digital filter waveforms were plotted and output was verified.

EXNO: 4 b

DESIGN OF HIGH PASS FIR DIGITAL FILTERS

AIM:

To generate design of high pass FIR digital filter using SCILAB

APPARATUS REQUIRED:

PC Installed with SCILAB

PROGRAM

Algorithm to Design FIR HPF Filter Using Window Techniques

Define Specifications

❖ Input the desired filter specifications:

- ✧ f_c : Cutoff frequency (normalized, $0 < f_c < 0.5$)
- ✧ M : Filter order (number of taps - 1)
- ✧ T : Sampling period (seconds) - optional for normalized frequency.

❖ Calculate the Ideal Filter Coefficients:

- Calculate the ideal impulse response $h_d[n]$ for the desired high-pass filter:
 - For $n = 0, 1, 2, \dots, M$:
 - If $n = \frac{M}{2}$:
 - $h_d[n] = 1 - 2f_c$
 - Else:
 - $h_d[n] = -\frac{\sin(2\pi f_c(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})}$

❖ Choose the Window Function

- Select the window function:
 - Rectangular window: $w[n] = 1$
 - Hamming window: $w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M}\right)$
 - Hanning window: $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{M}\right)\right)$
 - Bartlett window: $w[n] = \frac{2}{M} \left(\frac{M}{2} - \left|n - \frac{M}{2}\right|\right)$

❖ **Apply the Window Function:**

Multiply the ideal impulse response $hd[n]$ by the chosen window function $w[n]$ to get the actual filter coefficients $h[n]$: $h[n] = hd[n] \cdot w[n]$

❖ **Output the FIR Filter Coefficients:**

The final filter coefficients $h[n]$ represent the FIR filter.

RESULT:

Thus design of High pass FIR digital filter waveforms were plotted and output was verified.

AIM:

To generate design of band pass FIR digital filter using SCILAB

APPARATUS REQUIRED:

PC Installed with SCILAB

PROGRAM**Algorithm to Design FIR BPF Filter Using Window Techniques****Define Specifications**❖ **Input the desired filter specifications:**

- ❖ f_{c1} : Lower cutoff frequency (normalized, $0 < f_{c1} < 0.5$)
- ❖ f_{c2} : Upper cutoff frequency (normalized, $0 < f_{c2} < 0.5$)
- ❖ M : Filter order (number of taps - 1)
- ❖ T : Sampling period (optional, only if frequencies are not normalized)

❖ **Calculate the Ideal Filter Coefficients:**

- Calculate the ideal impulse response $h_d[n]$ for the desired band-pass filter:
 - For $n = 0, 1, 2, \dots, M$:
 - If $n = \frac{M}{2}$:
 - $h_d[n] = 2(f_{c2} - f_{c1})$
 - Else:
 - $h_d[n] = \frac{\sin(2\pi f_{c2}(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})} - \frac{\sin(2\pi f_{c1}(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})}$

❖ **Choose the Window Function**

- Select the window function:
 - Rectangular window: $w[n] = 1$
 - Hamming window: $w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M}\right)$
 - Hanning window: $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{M}\right)\right)$
 - Bartlett window: $w[n] = \frac{2}{M} \left(\frac{M}{2} - \left|n - \frac{M}{2}\right|\right)$

❖ **Apply the Window Function:**

Multiply the ideal impulse response $hd[n]$ by the chosen window function $w[n]$ to get the actual filter coefficients $h[n]$: $h[n] = hd[n] \cdot w[n]$

❖ **Output the FIR Filter Coefficients:**

The final filter coefficients $h[n]$ represent the FIR filter.

RESULT:

Thus design of band pass FIR digital filter waveforms were plotted and output was verified.

EXNO: 4 d

DESIGN OF BAND STOP FIR DIGITAL

FILTERSAIM:

To generate design of band stop FIR digital filter using SCILAB

APPARATUS REQUIRED:

PC Installed with SCILAB

PROGRAM

Algorithm to Design FIR BSF Filter Using Window Techniques

Define Specifications

❖ Input the desired filter specifications:

- ✧ f_{c1} : Lower cutoff frequency (normalized, $0 < f_{c1} < 0.5$)
- ✧ f_{c2} : Upper cutoff frequency (normalized, $0 < f_{c2} < 0.5$)
- ✧ M : Filter order (number of taps - 1)
- ✧ T : Sampling period (optional, only if frequencies are not normalized)

❖ Calculate the Ideal Filter Coefficients:

- Calculate the ideal impulse response $h_d[n]$ for the desired band-stop filter:
 - For $n = 0, 1, 2, \dots, M$:
 - If $n = \frac{M}{2}$:
 - $h_d[n] = 1 - 2(f_{c2} - f_{c1})$
 - Else:
 - $h_d[n] = \frac{\sin(2\pi f_{c1}(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})} - \frac{\sin(2\pi f_{c2}(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})}$

❖ Choose the Window Function

- Select the window function:
 - Rectangular window: $w[n] = 1$
 - Hamming window: $w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M}\right)$
 - Hanning window: $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{M}\right)\right)$
 - Bartlett window: $w[n] = \frac{2}{M} \left(\frac{M}{2} - \left|n - \frac{M}{2}\right|\right)$

❖ Apply the Window Function:

Multiply the ideal impulse response $hd[n]$ by the chosen window function $w[n]$ to get the actual filter coefficients $h[n]$: $h[n] = hd[n] \cdot w[n]$

❖ Output the FIR Filter Coefficients:

The final filter coefficients $h[n]$ represent the FIR filter.

RESULT:

Thus design of band stop FIR digital filter waveforms were plotted and output was verified.

EXNO: 5 SPEECH RECOGNITION USING SCILAB/MATLAB

DATE:

AIM:

To perform and verify speech recognition using SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

PROGRAM:

Algorithm for Speech Recognition

1. Collect and Preprocess Data:

- ❖ Record or Load Speech Data: Obtain audio recordings of speech.
- ❖ Preprocessing: Normalize the audio signal (e.g., adjust volume), remove noise if needed, and ensure consistent sampling rate.

2.Segmentation:

- ❖ Divide Audio into Frames: Split the audio signal into short overlapping frames (e.g., 20-40 ms long) to analyze speech segments effectively.

3.Feature Extraction:

Apply Window Function: Use a window function (e.g., Hamming or Hanning) to each frame to reduce edge effects.

Extract Features:

Mel-Frequency Cepstral Coefficients (MFCCs): Compute MFCCs for each frame to represent the speech signal in a compact form.

Other Features: Optionally extract other features like Linear Predictive Coding (LPC) coefficients, or formant frequencies.

4.Feature Normalization:

Normalize Features: Standardize or normalize the extracted features to ensure consistency across different recordings.

RESULT:

Thus the speech recognition using SCILAB was performed and verified.

EXNO: 6 SIMULATION OF MULTIRATE DSP WITHOUT FUNCTION

DATE:

AIM:

To perform and verify multirate DSP without function using SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

PROGRAM:

Down Sampling Algorithm

❖ Input Signal:

Let $x[n]$ be the input signal with a sampling rate of f_s .

❖ Determine Decimation Factor:

Define the decimation factor M , which determines how many samples to keep. For example, if $M=4$, you will keep every 4th sample.

❖ Low-Pass Filtering (Optional but Recommended):

Filter Design: To avoid aliasing artifacts, design a low-pass filter with a cutoff frequency of $f_{\text{cutoff}} = \frac{f_s}{2}$.

Apply Filter: Filter the signal $x[n]$ using the designed low-pass filter to get the filtered signal $y[n]$.

❖ Down sample the Signal:

Keep Every M -th Sample: From the filtered signal $y[n]$, create a new signal $x_{\text{dec}}[n]$ by keeping every M -th sample.

❖ Output:

The output signal $x_{\text{dec}}[n]$ is the downsampled version of the original signal.

Up Sampling Algorithm

❖ **Input Signal:**

Let $x[n]$ be the input signal with a sampling rate of f_s .

❖ **Determine Interpolation Factor:**

Define the Interpolation factor I which determines how many samples to discard. For example, if $M=4$, you will keep discard 4th sample.

❖ **Low-Pass Filtering (Optional but Recommended):**

Filter Design: To avoid aliasing artifacts, design a low-pass filter with a cutoff frequency of $f_{\text{cutoff}} = \frac{f_s}{2}$.

Apply Filter: Filter the signal $x[n]$ using the designed low-pass filter to get the filtered signal $y[n]$.

❖ **UP Sample the Signal:**

Discard Every M -th Sample: From the filtered signal $y[n]$, create a new signal $x_{\text{int}}[n]$ by discarding every I -th sample.

❖ **Output:**

The output signal $x_{\text{int}}[n]$ is the up sampled version of the original signal.

Result:

Thus the decimation process by a factor M and interpolation process by a factor L using SCILAB was implemented.

EX NO: 7 AUDIO EQUALIZATION TECHNIQUE

DATE:

AIM:

To perform and verify Audio Equalization Technique using SCILAB.

APPARATUS REQUIRED:

PC installed with SCILAB.

PROGRAM:

Algorithm for Audio Equalization

❖ **Input Audio Signal:**

Obtain the audio signal $x[n]$ which you wish to equalize.

❖ **Choose Equalizer Type:**

Graphic Equalizer: Equalizer with fixed frequency bands (e.g., 10 bands).

Parametric Equalizer: Equalizer with adjustable frequency bands, bandwidth, and gain for each band.

❖ **Design Equalizer Filters:**

Frequency Bands: Define the frequency bands for the equalizer (e.g., low, mid, high frequencies).

Filter Design: Design filters for each frequency band. This could involve:

Low-Pass Filters: For frequencies below a certain threshold.

High-Pass Filters: For frequencies above a certain threshold.

Band-Pass Filters: For specific frequency ranges.

Band-Stop Filters: To attenuate specific frequency ranges.

❖ **Apply Equalizer Settings:**

Adjust Gains: Set the gain for each frequency band based on the desired equalization settings. Gains can be positive (boost) or negative (cut).

Filter Application: Apply the designed filters to the audio signal with the specified gains.

❖ **Combine Filtered Signals:**

Filter Combination: Combine the output of each band-filtered signal according to the equalizer settings to get the final equalized signal.

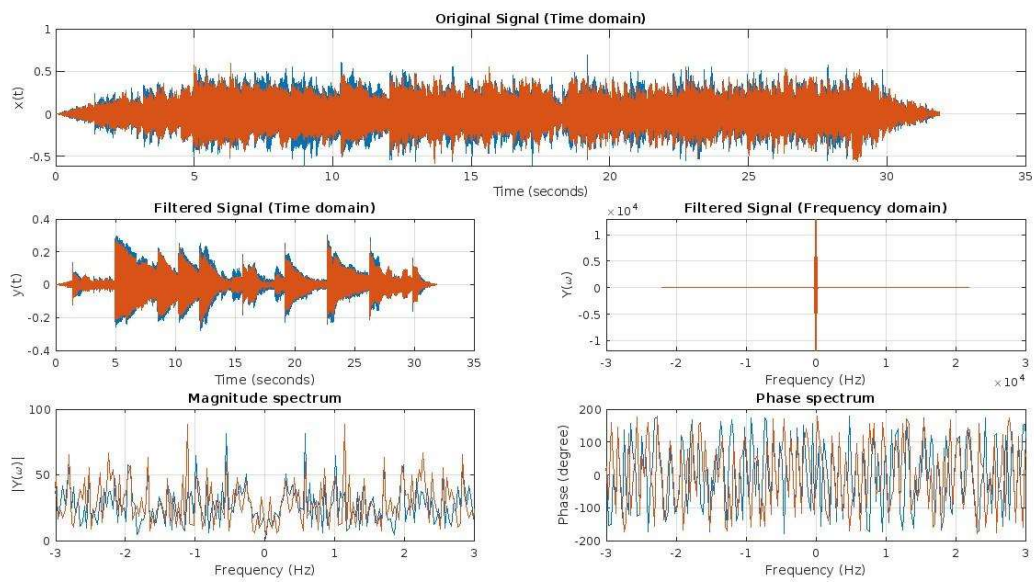
❖ **Output Equalized Signal:**

Result: The output signal $x_{eq}[n]$ is the equalized version of the original audio signal.

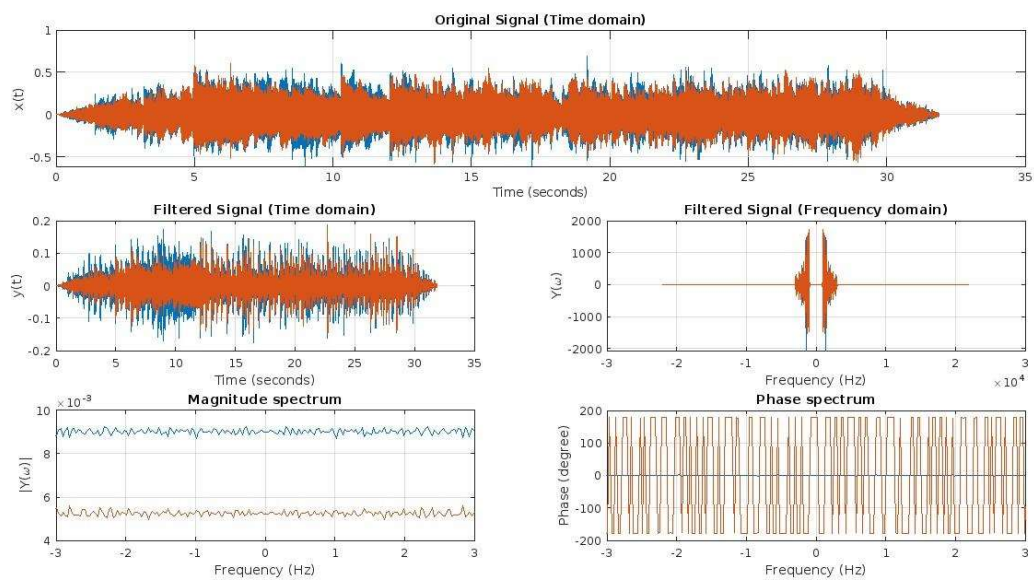
Detailed Steps

1. Obtain Audio Signal:
2. Load or record the audio signal $x[n]$ for processing.
3. Design Equalizer Filters:
4. Define Bands: Determine the cutoff frequencies for each band.
5. Create Filters: Design appropriate filters (e.g., FIR or IIR filters) for each band.
6. Apply Filters:
7. Filter Audio: Apply each filter to the audio signal $x[n]$ to isolate different frequency bands.
8. Adjust Gains:: Set Gain Values: Define the gain values for each frequency band according to the desired equalization effect.
9. Combine Filtered Signals:
10. Summation: Add the filtered signals, adjusting for gains, to form the equalized audio signal.
11. Output Equalized Signal:
12. Save or Play: Output the final equalized audio signal $x_{eq}[n]$ for listening or further processing.

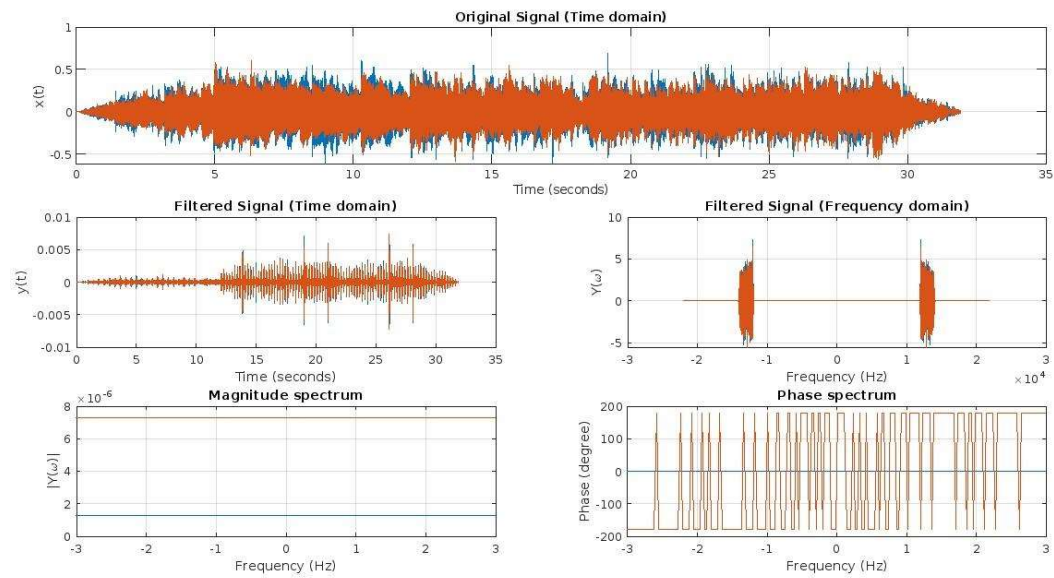
0-170hz



1-3khz

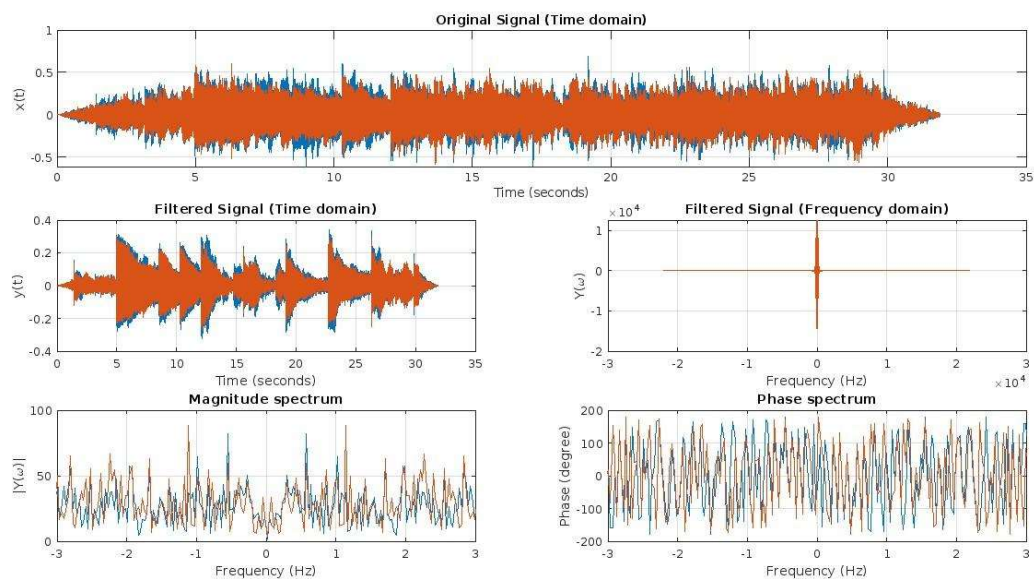


12-14khz

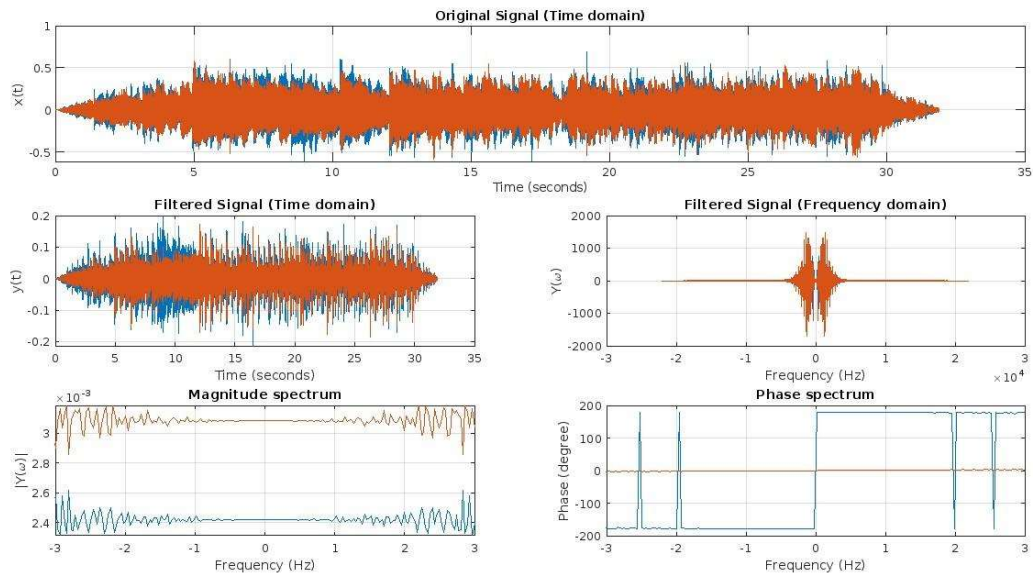


IIR filter Implementation-

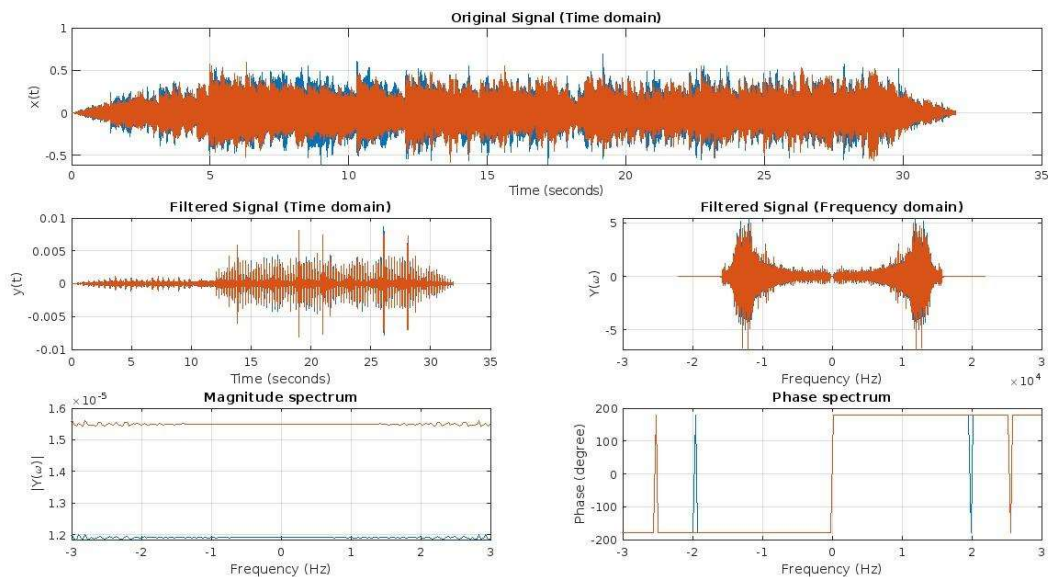
0-170hz



1-3khz



12-14khz



Result:

Thus the Audio Equalization Technique using SCILAB was implemented.

EXNO: 8 IMPLEMENTATION OF FIR, IIR FILTER IN DSP PROCESSOR.

DATE:

AIM:

To implement FIR and IIR filter in DSP processor.

APPARATUS REQUIRED:

PC installed with SCILAB.

DSP processor

PROGRAM:

Algorithm for Implementing FIR Filter on DSP Processor

❖ **Define Filter Specifications:**

Filter Order: Determine the order N of the FIR filter based on the design requirements.

Filter Coefficients: Obtain or design the FIR filter coefficients $h[n]$.

❖ **Load Filter Coefficients:**

Memory Allocation: Allocate memory on the DSP processor to store filter coefficients.

Coefficient Storage: Load the filter coefficients $h[n]h[n]h[n]$ into the allocated memory.

❖ **Prepare Input and Output Buffers:**

Input Buffer: Allocate memory to store the input signal samples.

Output Buffer: Allocate memory to store the filtered output signal samples.

❖ **Implement Filtering Operation:**

Convolution Operation: For each input sample, compute the convolution with the filter coefficients.

Accumulation: Accumulate the product of the filter coefficients and the corresponding input samples.

❖ **Update Buffers:**

Shift Operation: Shift the input samples and update the buffer for each new sample.

Store Output: Write the result of the convolution into the output buffer.

❖ **Output Filtered Signal:**

Signal Output: Output or further process the filtered signal stored in the output buffer

Algorithm for Implementing IIR Filter on DSP Processor

❖ Define Filter Specifications:

- ❖ Filter Order: Determine the order of the IIR filter.
- ❖ Filter Coefficients: Obtain or design the numerator (b) and denominator (a) coefficients of the IIR filter.

❖ Load Filter Coefficients:

- ❖ Memory Allocation: Allocate memory on the DSP processor for filter coefficients.
- ❖ Coefficient Storage: Load the numerator and denominator coefficients $b[n]$ and $a[n]$ into memory.

❖ Prepare Input and Output Buffers:

- ❖ Input Buffer: Allocate memory to store the input signal samples.
- ❖ Output Buffer: Allocate memory to store the filtered output signal samples.
- ❖ State Variables: Allocate memory for internal state variables (e.g., previous input and output samples).

❖ Implement Filtering Operation:

- ❖ Recursive Calculation: Use the difference equation to compute the filter output.
- ❖ Update State: Update state variables with the current input and output values.

❖ Update Buffers:

- ❖ Store Output: Write the result of the filter calculation into the output buffer.
- ❖ Update Input Buffer: Update the input buffer for the next sample.

❖ Output Filtered Signal:

Signal Output: Output or further process the filtered signal stored in the output buffer.

Result:

Thus the implementation of FIR and IIR filter in DSP processor using SCILAB was implemented.